```
from behave import given, when, then

@given('I am a Gold Frequent Flyer member')
def step_impl(context):
    assert True
```

Behave uses a different notation when passing parameters in to step definitions. Although Behave can be configured to use regular expressions, by default it uses named variables, a little like JBehave:

```
@given('I am a {status} Frequent Flyer member')
def step_impl(context, status):
    assert True
```

Extract the status from the step text.

Inject status into the step definition as a parameter.

Naturally, you can pass in more than one parameter:

```
@given('the flying distance between {departure} and {destination} is
     {distance} km')
def step_impl(context, departure, destination, distance):
    assert True
```

Behave will automatically convert basic parameter types such as numbers, percentages, dates, and times. You can also, with a little extra effort, configure automatic type conversions for the parameters that you pass in to the step definitions.

6.6.4 Combining steps

It's often useful to be able to combine several steps into a single step in order to avoid repetition and to simplify the scenarios. For example, suppose you already implemented the following two scenario steps:

```
Given Joe is a Frequent Flyer member
Given Joe logs in
```

In some scenarios, it would be useful to combine these two steps into a single one:

```
Given a Frequent Flyer member Joe has logged in
```

You could implement this step in Behave by reusing the previous two step definitions as shown here:

```
@given('a Frequent Flyer member {name} has logged in')
def step_impl(context,name):
    context.execute_steps(u"""
        Given {name} is a Frequent Flyer member
        And {name} logs in
    """)
```

Reuse the previous steps.

This is a useful trick, as it encourages step reuse, limits duplication, and makes it easier to write shorter, more concise scenarios.

6.6.5 Step definitions using embedded tables

Like most of the other tools discussed here, Behave lets you pass tabular parameters to your step definitions. Suppose you're implementing the following scenario using Behave: